

Models And Agents Report

Date: 2026-03-28

Scope

This report lists the database models, AI agents, and core service layers that were created or adjusted during the current build phase while moving the platform toward a case-centric multi-agent legal AI architecture.

Database Models Created

1. 'VoiceRecording'

File: 'backend/models/voice_recording.py'

Purpose:

- stores uploaded or browser-recorded voice intake artifacts
- links each recording to a case and tenant
- preserves transcript status, transcript text, and transcript metadata

Key fields:

- 'filename'
- 'storage_path'
- 'mime_type'
- 'file_size'
- 'transcription_status'
- 'transcript_text'
- 'transcript_language'
- 'case_id'
- 'tenant_id'

Why it matters:

- this model enables client call capture, transcription persistence, and later transcript reasoning

2. 'ConsultationRequest'

File: 'backend/models/consultation_request.py'

Purpose:

- stores structured intake information extracted from client speech or form submission
- supports consultation booking and case-description workflows
- bridges public/client intake with internal legal operations

Key fields:

- 'client_name'
- 'client_email'
- 'client_phone'
- 'booking_intent'
- 'urgency_level'
- 'legal_area'
- 'preferred_schedule'

- 'issue_summary'
- 'extracted_case_description'
- 'public_reference'
- 'source_channel'

Why it matters:

- this is the main workflow model for turning raw voice intake into actionable legal intake data

Database Models Adjusted

3. 'Case'

File: 'backend/models/case.py'

Adjustments:

- added 'voice_recordings' relationship
- added 'consultation_requests' relationship

Why it matters:

- the system is now truly case-centric across documents, voice, and intake workflows

AI Agent Layer Created

4. 'BaseAgent' and 'AgentResult'

File: 'backend/services/ai/agents/base_agent.py'

Purpose:

- defines a shared contract for agent execution
- standardizes 'payload', 'warnings', 'error', and 'trace'

Why it matters:

- this is the foundation for a consistent multi-agent architecture

5. 'SummarizationAgent'

File: 'backend/services/ai/agents/summarization_agent.py'

Purpose:

- improves document summaries using a provider-agnostic LLM gateway
- keeps a structured JSON output for downstream intelligence

Used by:

- 'backend/services/ai/summarization_service.py'

6. 'IntakeAgent'

File: 'backend/services/ai/agents/intake_agent.py'

Purpose:

- converts transcripts into structured legal intake payloads
- normalizes client identity, booking intent, urgency, and issue details

Used by:

- 'backend/api/consultations.py'
- 'backend/api/public.py'

7. 'RetrievalAgent'

File: 'backend/services/ai/agents/retrieval_agent.py'

Purpose:

- performs hybrid retrieval using lexical and semantic search
- merges and ranks chunk results for grounded question answering

Used by:

- 'backend/services/ai/rag_service.py'

8. 'CaseReasoningAgent'

File: 'backend/services/ai/agents/case_reasoning_agent.py'

Purpose:

- synthesizes the full case picture using documents, consultation requests, and voice transcripts
- produces structured outputs for:
 - overview
 - narrative summary
 - main issues
 - key dates
 - legal risks
 - recommended next steps

Used by:

- 'backend/services/ai/copilot_service.py'

9. 'VerifierAgent'

File: 'backend/services/ai/agents/verifier_agent.py'

Purpose:

- checks whether generated answers are adequately supported by retrieved evidence
- can fall back to a more grounded supported answer when verification is weak

Used by:

- 'backend/services/ai/rag_service.py'

Why it matters:

- this is the start of the trust/proof-first layer in the architecture

Core AI Services Adjusted

10. 'LLMGateway'

File: 'backend/services/ai/llm_gateway.py'

Purpose:

- centralizes provider configuration
- supports OpenAI-compatible endpoints such as OpenRouter

Configuration source:

- 'backend/core/config.py'

11. 'SummarizationService'

File: 'backend/services/ai/summarization_service.py'

Adjustment:

- now uses 'SummarizationAgent' first, then falls back safely

12. 'RagService'

File: 'backend/services/ai/rag_service.py'

Adjustments:

- now uses 'RetrievalAgent'
- now uses 'VerifierAgent'
- remains the grounded QA orchestration layer

13. 'CopilotService'

File: 'backend/services/ai/copilot_service.py'

Adjustments:

- now uses 'CaseReasoningAgent' for case summarization and case risk analysis
- now pulls from consultation requests and voice recordings, not just documents

Architecture Status

Current explicit agent stack:

- 'SummarizationAgent'
- 'IntakeAgent'
- 'RetrievalAgent'
- 'CaseReasoningAgent'
- 'VerifierAgent'

Current architectural shape:

- voice intake and transcript persistence
- consultation workflow extraction
- hybrid retrieval and RAG
- case-centric reasoning
- grounded verification
- separate internal app and client portal

Provider Status

The codebase supports OpenAI-compatible providers through:

- 'OPENAI_API_KEY'

- 'LLM_BASE_URL'
- 'LLM_MODEL'
- 'SUMMARY_AGENT_MODEL'

Important current note:

- the local environment currently has an API key present
- but 'LLM_BASE_URL' and 'LLM_MODEL' were not yet configured in '.env' at the time of inspection
- that means the OpenRouter-style provider path is supported by code, but not fully activated in the local runtime yet

Recommended runtime values:

- 'LLM_BASE_URL=https://openrouter.ai/api/v1'
- 'LLM_MODEL=openai/gpt-4o-mini'
- 'SUMMARY_AGENT_MODEL=openai/gpt-4o-mini'

Summary

The platform has moved from a backend prototype with AI features into an actual early multi-agent legal AI system. The most important structural changes were the creation of case-linked voice and consultation models, the standardization of the agent contract, and the addition of reasoning and verification agents that begin to reflect the architecture promised in the project vision.